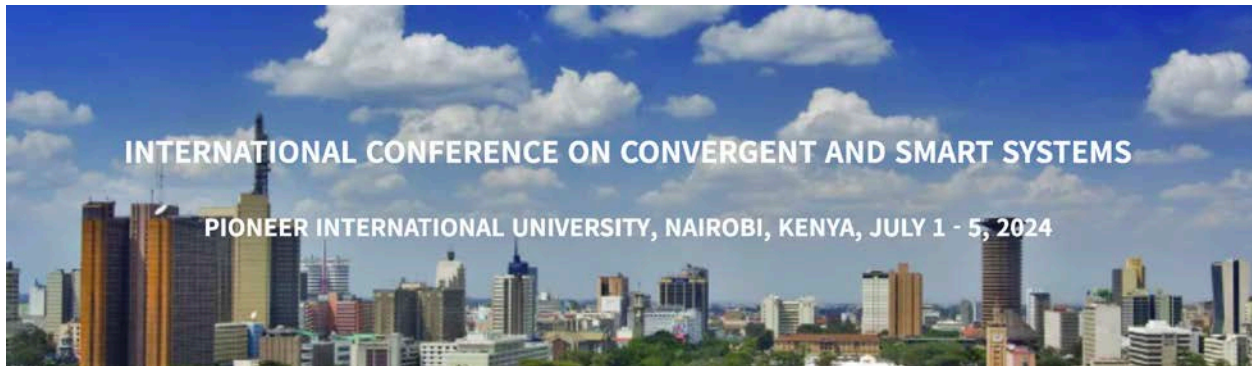


2024 INTERNATIONAL CONFERENCE ON CONVERGENT AND SMART SYSTEMS (ICCSS 2024)

PIONEER INTERNATIONAL UNIVERSITY, NAIROBI, KENYA,
JULY 1 - 5, 2024

Proceeding



SPONSORED BY



**CHOSUN
UNIVERSITY**



사단법인 한국차세대컴퓨팅학회
Korean Institute of Next Generation Computing



**PIONEER
INTERNATIONAL
UNIVERSITY**

Enhancing Windows Malware Classification Accuracy Using SHAP-Based Feature Selection in Ensemble Classifiers

Stuart Asiimwe

Department of Computer Science and
Software, Hanyang University
Seoul, South Korea
stuart@hanyang.ac.kr

Eul Gyu Im

Department of Computer Science and
Software, Hanyang University
Seoul, South Korea
imeg@hanyang.ac.kr

ABSTRACT

Malware continues to pose a significant threat to computer security, with Windows operating systems being the most common target. Furthermore, traditional classification methods struggle to keep pace with the evolving complexity of malware. In this study, we propose a framework that employs SHAP (SHapley Additive exPlanations) values for feature selection within ensemble classifiers, with the aim of improving the accuracy of malware classification while maintaining model transparency. The proposed study boosted a significant improvement in accuracy, with our ensemble model reaching 98.9% accuracy, outperforming both the baseline ensemble's accuracy of 95.3% and the average accuracy of individual classifiers, which stood at 91.7%.¹

KEYWORDS

Malware Analysis, Ensemble Learning, Machine Learning, Explainable AI (XAI)

1. INTRODUCTION

Over the past decade, the extensive study of the range of threats that malware poses to computer security continues to be of importance, with Windows operating systems often being the primary focus [17]. When it comes to Malware analysis, three major approaches have been involved, that is to say: Static analysis, dynamic analysis and hybrid analysis, which is a combination of both static and dynamic analysis of the malware attributes [3]. However, these traditional detection and classification methods are continuously finding it a challenge to match the ever-evolving complexity of malware.

In order to combat this issue of the constantly evolving malware, most of the studies have been using various machine learning algorithms to detect and classify malware quickly and accurately [11]. In some recent studies, API call sequences are mostly used as a feature to detect malware.

Practically, many features that can be extracted from malware samples, and features used for malware analysis constantly evolve as new technology and analytic methodologies emerge [7, 16, 22].

Researchers and analysts are continually discovering new traits and behaviors that may be important in malware analysis and classification [4, 18, 23]. That being stated, not all of these features may be equally useful in developing successful classification models [4]. The difficulty is identifying the most relevant and non-discriminative features for the task at hand.

In the light of the above literature review and more, we conducted comprehensive research, using the PE (Portable Executable) Header Malware Dataset [14] and we suggested a framework that utilizes SHAP values [12] for feature selection in ensemble classifiers, using tree-based models (Decision Trees and Random Forest) and gradient boosting methods such as AdaBoost, Gradient Boosting, and XGBoost. *Ensemble learning* has emerged as a powerful method for improving classification tasks by combining several learning algorithms. However, Machine Learning models being at their core, these models are often black boxes that makes their interpretation difficult. This then results into trust issues and deployment challenges due to a lack of transparency in these models. In order to understand what the main features are, which features affect the output of the model, we need Explainable Machine Learning techniques that unravel some of these aspects. One of these techniques is the SHAP method, used to explain how each feature affects the model, and allows local and global analysis for the dataset and problem at hand, hence improving interpretability.

Our work seeks to address some of the above complexities surrounding malware classification tasks, and the following research questions were answered in this research work:

- (1) What is the impact of using ensemble learning in malware classification?
- (2) How does the utilization of SHAP values to guide feature selection methods impact malware classification accuracy?
- (3) To what extent does the application of data balancing techniques, like Synthetic Minority Over-sampling Technique (SMOTE), contribute to the effectiveness of model's malware classification?

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

- (4) How does usage of the visualized SHAP-selected features contribute to understanding of the machine learning model's decision-making process?

2. RELATED WORKS

This section covers a review of various approaches and strategies on malware detection and classification.

Researchers have explored various feature selection methods to improve the accuracy of malware classification. Xiaofeng [21] used a combined deep learning and machine learning model for malware behavior analysis with binary classification and Random Forest (RF). The combination of RF and Long Short-Term Memory (LSTM) model classified malware with 96.7% accuracy.

In addition, Han et al. [8] used RF, Decision Tree (DT), k-nearest neighbor, and XGBoost methods to detect and classify multi-class malware by combining static and dynamic API call sequences. Using their MalDAE framework, their study clarified the differences and relationships between API sequences as well as the many forms of malicious behavior. Their work achieved a 97.8% detection accuracy and 94.4% classification accuracy with RF. Bahtiyar et al. [9] proposed a multidimensional machine learning approach to detect advanced malware. They applied various regression models by using five distinguished features: stealthiness, Stuxnet closeness, behavioral instability, conventional malware arsenal and metamorphic engine. The proposed method reported 0.0001 mean squared error rate.

3. COMPARATIVE ANALYSIS

As the volume of cyber-attacks with advanced malware increases, it makes malware hard for detection and classification. Ensemble classifiers, which combine multiple base classifiers, have shown promise in improving malware detection accuracy [2]. Studies have investigated the effectiveness of random forests, gradient boosting, and voting ensembles for identifying malicious software. In addition, the use of SHAP values has gained attention in the field of machine learning interpretability [5]. SHAP values provide insights into feature importance and allow practitioners to understand how individual features contribute to model predictions. Researchers have applied SHAP-based techniques to enhance transparency in malware classification models [15].

Comparative studies have evaluated the trade-off between accuracy and transparency in malware classifiers [6]. These investigations compare ensemble models (such as bagging and stacking) with traditional classifiers (e.g., SVM, k-NN, and decision trees). Understanding the impact of feature selection methods on both accuracy and interpretability is crucial. In fact, Lundber and Lee argued that understanding why a model makes a certain prediction is crucial, especially when dealing with complex models like ensemble or deep learning models [12]. The SHAP framework [1] is instrumental in this regard, as it quantifies the contribution of each feature to a particular prediction, offering valuable insights into the decision-making process of the model.

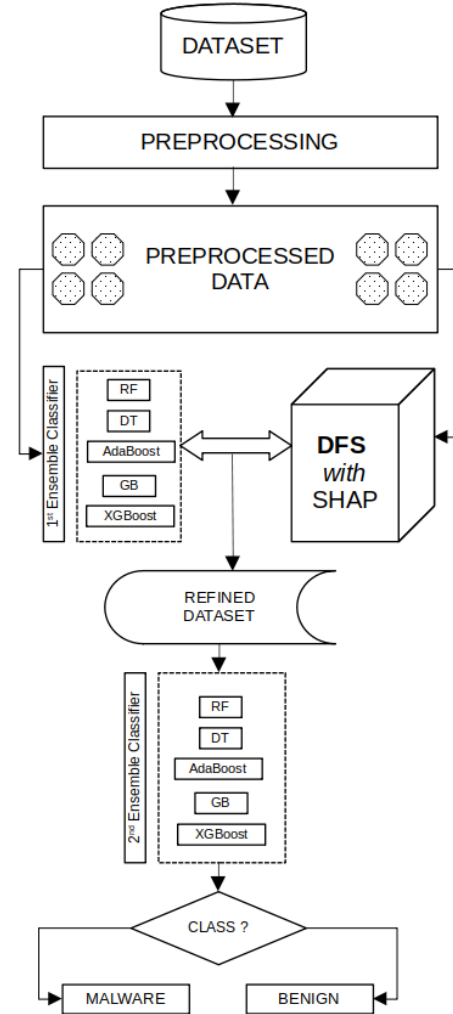
4. EXPERIMENTAL AND COMPUTATIONAL DETAILS

4.1. Workflow of the Proposed Framework

In our proposed framework, the experiment started with the collection and preprocessing of the dataset [14]. Following this step, we constructed the ensemble classifier by using tree-based models and gradient boosting techniques such as Decision Trees (DT), Random Forest (RF), AdaBoost, Gradient Boosting (GB), and eXtreme Gradient Boosting (XGBoost), as shown in Figure 1.

Once the base classifiers were trained, they were used to construct the first Meta/Ensemble Classifier. During this phase, an evaluation was conducted while simultaneously computing the mean absolute SHAP value for each feature across all samples

Figure 1: The Proposed Workflow



using the Dynamic Feature Selection (DFS) Mechanism proposed in this work. The DFS algorithm facilitated the selection

of features based on their SHAP values, proceeding until our predetermined threshold of viable features was reached.

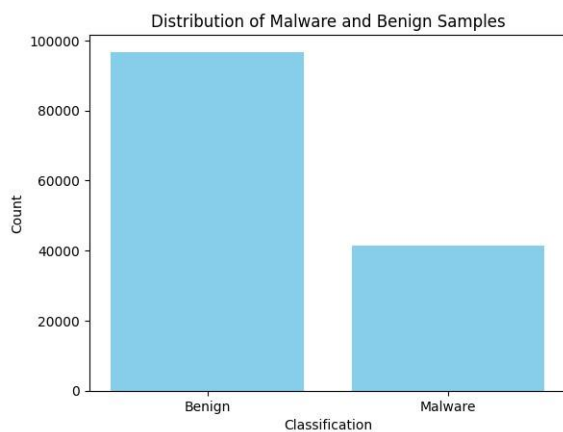
The results were then sorted in descending order, that is to say, from biggest (most influential) to the smallest (least influential) features, as described in detail under section 4.6. These selected features were then used as a guide or filter to preprocess our initial dataset resulting in the creation of a 'Refined Dataset' or DataFrame. It is important to note that this refined DataFrame represented a distilled version of the original dataset, containing only the most influential features identified through the DFS mechanism. Subsequently, the refined DataFrame was used to train the new second Ensemble Classifier. The performance of this meta-classifier was carefully monitored and evaluated on the testing data, and the results were analyzed and documented as discussed in detail under Section 5.

4.2. The Dataset and Preprocessing

The dataset selected for this study is the *Windows PE Malware Dataset* [14] which comprises 96,724 Benign samples and 41,323 Malware samples. This dataset is composed of static features that are derived from Windows Portable Executable (PE) files. These features are obtained through static analysis, which does not require the execution of the binaries. The dataset includes a variety of PE file properties such as headers, sections, and metadata. These properties are essential for the task of classifying malware, as they provide insights into the characteristics that are often manipulated by malware to evade detection or perform malicious activities.

However, despite our total size of the dataset, which is more than 130,000 samples, the dataset exhibits a class imbalance, as shown in Figure 2, with significantly more benign samples than the malicious ones.

Figure 2: Distribution of malware and benign samples in the dataset

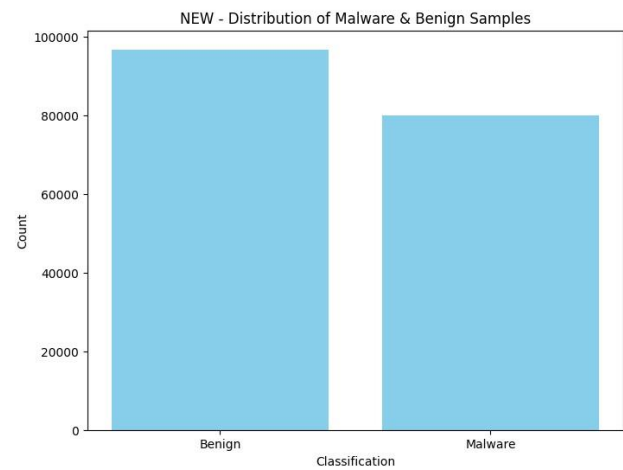


To address this imbalance and ensure fair representation of both classes during training, the dataset had to be preprocessed using the Synthetic Minority Over-sampling Technique (SMOTE) [10] to generate synthetic samples for the minority class (i.e. malware samples). By synthetically creating new malware samples, the

dataset was relatively rebalanced to achieve a more equitable distribution between malware and benign samples. We preprocessed the dataset, using a custom ratio of 0:97724, 1:80000 as the sampling strategy, where '0' represents the benign and '1' malware samples respectively, as opposed to using "Auto" mode of the SMOTE algorithm as the default sampling strategy.

The preprocessed dataset now consists of approximately 96,724 benign samples and 80,000 malware samples, as illustrated in Figure 3. This balanced dataset enables the classifiers to learn from a more representative sample distribution, leading to improved model performance in terms of generalization.

Figure 3: New Distribution of malware and benign samples in the dataset after applying the SMOTE algorithm



Additionally, as part of the pre-processing stage, some features, such as the column "md5", were removed from the dataset because they were not relevant to this classification task and would potentially introduce noise into the model. Furthermore, to guarantee the accuracy and integrity of the data used to train the machine learning models, missing values were handled appropriately using methods such as imputation or elimination. Thus in order to effectively prepare the dataset for model training and evaluation, some of these preprocessing steps were essential.

4.3. Base Classifiers selection

To enhance the robustness of our malware classification task, a diverse set of base classifiers has been carefully selected. These include Random Forest (RF), Decision Tree (DT), Gradient Boosting (GB), AdaBoost and XGBoost. Each of these classifiers offers unique strengths and is adept at capturing different aspects of the underlying data distribution.

4.4. Constructing of the Ensemble Classifiers

The meta or ensemble classifier is built by combining predictions from numerous base or classifiers (weak learners) to reach a final judgment [21]. This method uses the diversity of individual classifiers to increase overall performance. The ensemble classifier can be generated using many ensemble

techniques, such as Voting, Stacking, Boosting or Bootstrap aggregating, also commonly known as Bagging. The choice of ensemble technique depends on the characteristics of the dataset and the performance goals [3].

In this work, we used the stacking technique to build our meta classifiers. Stacking, also known as stacked generalisation, is the process of integrating predictions from different base classifiers as features and feeding them into a meta-classifier, which learns to create the final prediction. This technique enables us to take advantage of the various views of individual classifiers and their complimentary qualities.

The *StackingClassifier* from the *scikit-learn* library was utilized to implement the ensemble classifier. This classifier requires the specification of a list of tuples, where each tuple contains an identifier for the base classifier and the corresponding classifier object itself. Additionally, to combine the predictions of the base classifiers, a final estimator, also called the meta-classifier, was used.

4.5. Implementing SHAP Algorithm

Several interpretability techniques exist, to mention but a few, Feature Importance Scores, Partial Dependence Plots, and LIME (Local Interpretable Model-agnostic Explanations) [19]. However, the prevalent inconsistency among current feature attribution methods for tree ensembles poses a prominent concern [13]. This inconsistency poses a challenge as it may misrepresent the importance of features when model adjustments occur; like in the case of our meta-classifiers.

Therefore, in order to gain insights into the importance of each feature in the predictions of our meta-classifier, we chose to employ the SHAP (SHapley Additive exPlanations) algorithm. These (SHAP) values, rooted in game theory concepts [24], offer a unified and consistent approach to feature attribution. Notably, research by Lundberg and Lee (2017) demonstrates that SHAP values are the only method that consistently provides accurate and locally interpretable results while adhering to essential properties such as missingness and conditional dependence measurement. This robustness makes SHAP values an ideal choice for figuring out which features matter most in our ensemble models. The SHAP value of feature i for the model f is represented by $\phi_i(f)$ and was calculated using the following equation:

$$\phi_i(f) = \sum_{S \subseteq \{1, \dots, M\} \setminus \{i\}} \frac{M!}{|S|!(M - |S| - 1)!} [f(S \cup \{i\}) - f(S)]$$

In the given equation, $\phi_i(f)$ represents the SHAP value of feature i for the model f , indicating the contribution of feature i to the model's prediction. In other words $\phi_i(f)$ captures the difference in model output caused by including the feature i compared to excluding it. The variable M denotes the total number of features in the dataset, defining the size of the feature space, while S represents a subset of features selected for evaluation. The function $f(S)$ signifies the model's output when considering only

the features in subset S , aiding in assessing the impact of specific feature subsets on the model's overall prediction.

4.6. The Dynamic Feature Selection (DFS) Mechanism

This section outlines a data-driven approach to selecting features for malware classification using the developed DFS algorithm. Our goal was to improve the precision of our ensemble model through feature engineering [20], that is to say, by using the most influential features that were adaptively selected based on their SHAP values. In order to ensure efficient implementation and scaling, we did take into account the computational resources and complexity involved in evaluating recalculated SHAP values for dynamic feature selection.

4.6.1. Pseudocode for the DFS Mechanism

START

>Calculate the mean absolute SHAP value for each feature.

>Sort the features based on their mean SHAP values in descending order.

>Initialize an accumulated SHAP value.

>FOR each feature in the sorted list DO:

 >Add the mean SHAP value of the current feature to the accumulated SHAP value.

 >IF accumulated SHAP value exceeds the threshold OR maximum number of features is reached THEN

 >Exit loop.

 >ELSE

 >Add the current feature index to the list of selected features.

 >ENDIF

>END FOR

>Return the indices of the selected features.

STOP

The Steps involved in this DFS are as follows:

With the data prepared, the 1st meta-classifier is trained and tested. SHAP values are calculated for each feature using a TreeExplainer, such as RF, as in this study. After that, the mean absolute SHAP values are computed for each feature, and the features were sorted based on these values in descending order. This step helps identify the most influential features in the dataset. A subset of top features is selected based on a threshold or a maximum number of features.

After the selection of the top features based on their mean SHAP values, a refinement process takes place. These selected features serve as a filter, effectively refining the original dataset and retaining only the most influential attributes for further analysis. This process ensures that the dataset used for training the subsequent ensemble classifier is optimized, focusing on the features that only contribute the most to the predictive power of the model. The refinement of the dataset involves dropping the less important features, leaving us with a new streamlined dataset

referred to as the "*Refined Dataset*" in the proposed workflow model illustration in Figure 1.

This refined dataset is characterised by a reduced dimensionality compared to the original dataset, as it contains only the subset of features deemed most relevant for the classification task at hand. Once the refined dataset is prepared, it serves as the input for training the second ensemble classifier or the in our case the stacking classifier. The stacking ensemble classifier is initialized next, incorporating base classifiers (RF, DT, GB, AdaBoost and XGBoost). Then, the stacking ensemble classifier was trained using the refined dataset, and its performance was evaluated on the test data to assess its generalization ability.

5. RESULTS AND DISCUSSION

In this section, we present the results of our malware classification experiment using the DFS mechanism. We first discuss the performance metrics of individual base classifiers, followed by an analysis of the meta-classifier after combining the base classifiers. Additionally, we provide insights into the feature importance determined by the DFS mechanism and its impact on the classification performance.

5.1. Impact of Preprocessing with SMOTE

We investigated the impact of addressing dataset imbalance using the Synthetic Minority Over-sampling Technique on the performance of our classification model. This is because imbalanced datasets, where one class significantly outweighs the other, can lead to biased models that prioritize the majority class and overlook important patterns in the minority class, and such effects would affect the experimental results of our proposed work.

SMOTE is a widely used technique to alleviate this issue by synthesising new minority class instances (malicious samples), thus creating a more balanced dataset. We evaluate how the model's accuracy, precision, recall, and F1-score were affected. Figure 4 illustrates the comparative performance before and after SMOTE preprocessing. Figure 4a represents the model's performance metrics before SMOTE, while Figure 4b showcases the performance metrics after SMOTE.

The utilisation of SMOTE to tackle dataset imbalance led to consistent improvements in all performance metrics. For example, following the application of SMOTE, the Random Forest classifier demonstrated noticeable enhancements across various measures. Its accuracy rose from 91% to 95%, recall increased substantially from 77% to 94%, and the F1-score surged from 83% to 96%. While precision decreased slightly from 99% to 94%, it's important to recognize that this shift indicates a more conservative approach in the model's predictions. With the introduction of additional synthetic samples, the model learns from a broader set of instances rather than relying solely on direct matches. Therefore, this broader learning scope contributed to a slight reduction in precision.

Similar positive trends were observed in other classifiers like the Decision Tree, Gradient Boosting, AdaBoost, and XGBoost. These improvements show the effectiveness of SMOTE in mitigating the detrimental effects of class imbalance, thus enabling our classifiers to better discern the underlying data patterns,

resulting in more accurate predictions. Note however that, the slight decrease in precision for some classifiers like RF suggests that there may be an increase in false positives on the benign class, which is an expected and common trade-off when addressing class imbalance.

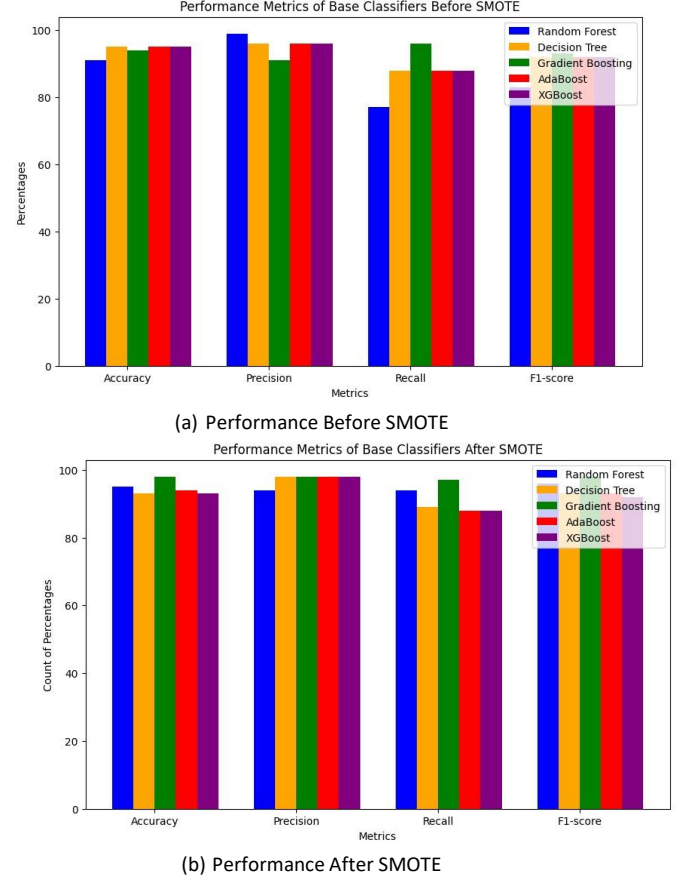


Figure 4: Illustration of the effect of Dataset preprocessing with the SMOTE Algorithm

5.2. Base Classifiers Performance

The results for before and after handling the dataset imbalance with SMOTE were recorded as shown in Table 1 and Table 2 respectively. Before addressing the class imbalance, the Random Forest classifier exhibited a high precision of 99%, indicating its ability to correctly classify most samples.

Table 1: Performance Metrics of Base Classifiers Before

Classifier	Accuracy	Precision	Recall	F1-score
Random Forest	0.91	0.99	0.77	0.83
Decision Tree	0.95	0.96	0.88	0.92
Gradient Boosting	0.94	0.91	0.96	0.93
AdaBoost	0.95	0.96	0.88	0.92
XGBoost	0.95	0.96	0.88	0.92

However, its recall was relatively low at 77%, suggesting that it struggled to identify all instances of malware. Similarly, the Decision Tree classifier showed a similar pattern, with high precision but lower recall, indicating a conservative approach in labeling samples as malware, as described in the previous section.

Table 2: Performance Metrics of Base Classifiers After SMOTE

Classifier	Accuracy	Precision	Recall	F1-score
Random Forest	0.95	0.94	0.94	0.96
Decision Tree	0.93	0.98	0.89	0.93
Gradient Boosting	0.98	0.98	0.97	0.98
AdaBoost	0.94	0.98	0.88	0.93
XGBoost	0.93	0.98	0.88	0.92

After applying the SMOTE algorithm, substantial improvements were observed in the recall of both Random Forest and Decision Tree classifiers. The Random Forest classifier’s recall increased to 94%, indicating a substantial improvement in identifying malware samples. This improvement suggests that the SMOTE algorithm helped the classifier to better recognize instances of malware, thereby reducing the risk of false negatives.

5.3. Ensemble Model Performance

When comparing the two ensemble models—one trained on the raw dataset and the other on the refined dataset from the DFS algorithm—the latter demonstrated an impressive 98% accuracy, which not only surpassed the 1st baseline ensemble model’s accuracy of 95% but also the average accuracy of individual classifiers (which initially stood at 91.7%) as shown in Table 3.

Table 3: Performance Metrics of the Ensemble Classifiers

Classifier	Accuracy	Precision	Recall	F1-score
1st Ensemble Classifier	0.95	0.96	0.98	0.92
2nd Ensemble Classifier	0.98	0.97	0.98	0.99

These experimental results highlight the effectiveness of our proposed method. The notable increase in classification accuracy successfully addresses our research questions, demonstrating how ensemble learning and the use of SHAP values can improve malware classification accuracy.

5.4. Interpretability and Insights

We utilized SHAP plots to visually represent the key features that were selected through a dynamic process by our DFS Algorithm as detailed under Section 4.6. SHAP provided global (Figure 5) and local interpretation methods based on aggregation of Shapley values obtained from the 1st Ensemble Model.

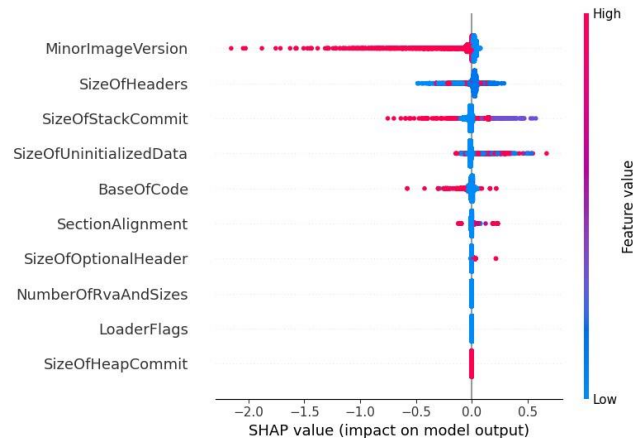
In this study, the summary plots show the average impact of each feature on model output across all samples in the test dataset (global) as seen in Figure 6. The features were ranked according to their mean absolute SHAP values, as shown in Table 4 for the top 10 best contributing features, allowing us to identify the most influential predictors.

Table 4: Mean Absolute SHAP Values Top 10 Features

Feature	Mean Absolute SHAP Value
ImageBase	1.9919
VersionInformationSize	1.2600
SectionsMaxEntropy	1.2205
SizeOfStackReserve	0.7289
ResourcesMinSize	0.5486
MajorLinkerVersion	0.5073
Checksum	0.4692
MajorOperatingSystemVersion	0.4230
Characteristics	0.4176
Subsystem	0.3365

The density scatter plot of SHAP values for each feature to identified how much impact each feature had on the model output as explained earlier in section 4.5. This plot combines feature importance with feature effects, that is to say, each point on the summary plot is a Shapley value for a feature and an instance. The Features are sorted by the sum of the SHAP value magnitudes across all samples. When the scatter points don’t fit on a line they pile up to show density, and the color of each point represents the feature value of that individual.

Figure 5: Global Feature Importance for the least Contributing Features



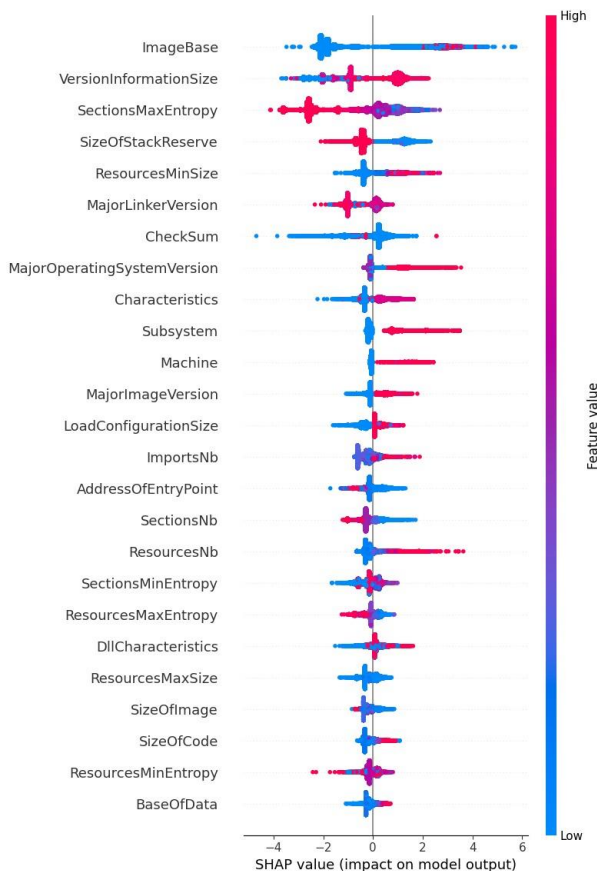
The position on the y-axis is determined by the feature and on the x-axis by the Shapley value. You can see from Figure 5 that the feature `SizeOfHeapCommit`, being the least important feature, has low Shapley values. Other features such as, `LoaderFlags`, `NumberOfRvaAndSizes` and `SizeOfOptionalHeader` also

had very low Shapley values thus a low impact on the model's output.

The features are ordered according to their importance. The colour represents the value of the feature from low(Blue) to high(Red). And for the case of our summary plot, the overlapping points were jittered in the y-axis direction, so we get a sense of the distribution of the Shapley values per feature. From Figure 6 we can see that **ImageBase** comes out as the top most contributing feature, confirming what was previously observed in the literature [18].

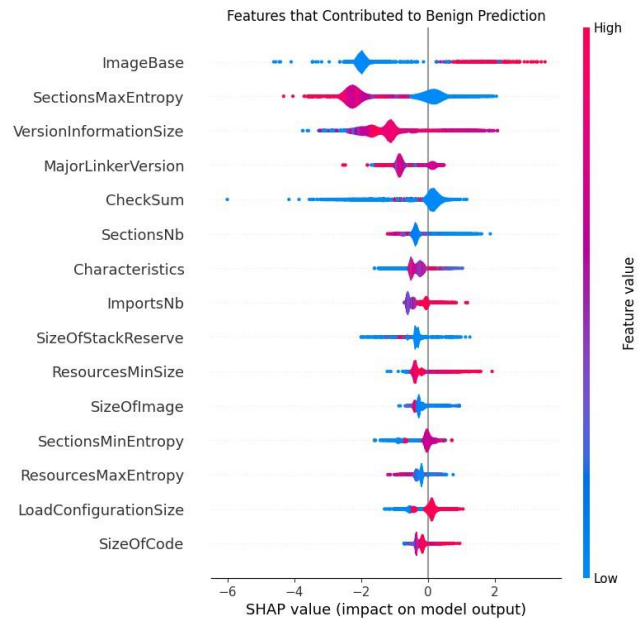
Figure 6: SOME OF THE TOP CONTRIBUTING FEATURES IN THE 1ST ENSEMBLE MODEL

Each dot is a single prediction (row) from the dataset. The x-axis is the actual value from the dataset while the y-axis is the SHAP value for that feature, which represents how much knowing that feature's value changes the output of the model for that sample's prediction. The color corresponds to a second feature that may have an interaction effect with the feature we are plotting (by default this second feature is chosen automatically). If an interaction effect is present between this other feature and the feature we are plotting, it will show up as a distinct vertical pattern of coloring.



To gain further insight, we took a closer look at how different features contribute to classifying instances as either benign or malicious. Figures 7 and 8 show this breakdown. These plots helped us understand which specific features were most influential in distinguishing between the two classes. This insight was key in guiding and fine-tuning the feature isolation process of the DFS, resulting into creation of a highly effective refined dataset.

Figure 7: Features that were important in classification of a row as Benign



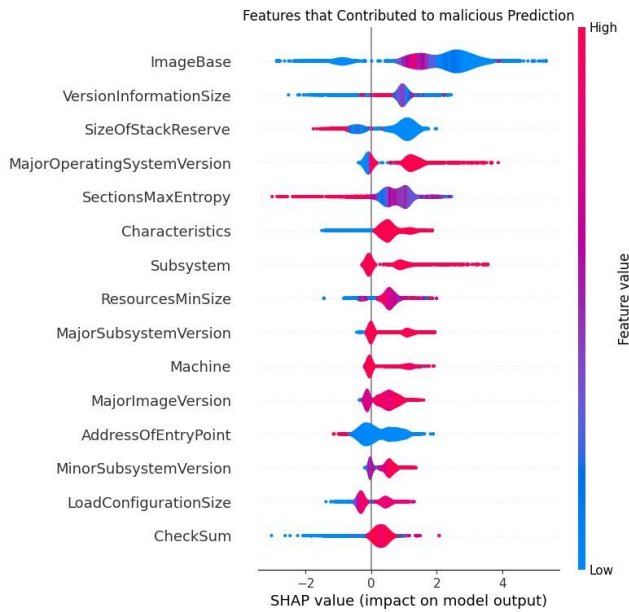
In Figure 7, the analysis reveals that features such as **ImageBase**, **SectionMaxEntropy**, and **VersionInformationSize** greatly influenced the model's decision-making process. This observation holds merit due to the nature of these features. For instance, **ImageBase** signifies the preferred base address where the executable file is loaded into memory. Anomalies or discrepancies in this value, along with conflicts with other loaded modules, often indicate malicious activities such as DLL injection or memory manipulation techniques commonly associated with malware.

Another instance is the feature **SectionsMinVirtualsize** which stands out as an important contributor to the model's decision-making process for classifying a sample as Legitimate Malware, as depicted in Figure 8. This attribute represents the minimum virtual size of sections in the executable file, providing insights into the organization and structure of potentially malicious software. Understanding variations or abnormalities in this feature can offer valuable clues regarding the presence of harmful behavior, aiding in the accurate detection and classification of malicious entities.

5.5. Impact of the DFS and the Refined Dataset

From the initial pool of 57 features, three columns i.e. *md5*, *name*, and the target column *Legitimate* were dropped during the initial preprocessing stage. Following this, the remaining 54 features underwent further filtration by the DFS algorithm, which assessed their importance for the classification task.

Figure 8: Features that were important in classification of a row as Malicious



Those identified as most important, as elaborated in subsection 5.4, were then utilized to refine the original dataset. This refinement process culminated in the creation of the 'Refined DataFrame'. The results of the improved metrics were as illustrated in Table 5.

Table 5: Performance Metrics for Base Classifiers From Refined Dataset

Classifier	Accuracy	Precision	Recall	F1-score
RF Classifier	0.9859	0.9945	0.9865	0.9855
GB Classifier	0.9942	0.9936	0.9936	0.9936
DT Classifier	0.9937	0.9934	0.9926	0.9930
AdaBoost Classifier	0.9965	0.9949	0.9974	0.9961
XGB Classifier	0.9868	0.9856	0.9873	0.9865

The performance metrics of the base classifiers, utilising the refined dataset generated by the DFS mechanism, underscore their efficacy in capturing relevant information for classification tasks. This was indicated by generally high accuracy, precision, recall, and F1-score values across the models. Specifically, the classifiers achieved accuracy values ranging from 98.68% to 99.65%, with corresponding precision, recall, and F1-score values demonstrating consistent high performance.

These results suggest that the base classifiers exhibit strong predictive capabilities even before refining the dataset. However,

with further enhancement through the DFS mechanism, the classifiers demonstrated even better performance, leading to more accurate and reliable predictions.

5.6. Contributions

- 5.6.1. *Improved Classification Accuracy:* Our method boosted better accuracy which outperformed baseline and individual classifiers.
- 5.6.2. *Dynamic Feature Selection:* We introduced a novel algorithm that uses SHAP values for adaptive feature selection, enhancing model performance and interpretability.
- 5.6.3. *Interpretability and Insights:* Visualization of SHAP values for features in malware dataset provided deeper insights into the model's decision-making process.
- 5.6.4. *Bridging Ensemble Learning with SHAP Values:* We integrated SHAP-based feature selection into ensemble classifiers, bridging two important areas of machine learning.
- 5.6.5. *Balancing Accuracy with Interpretability:* We addressed the challenge of reconciling accuracy and model interpretability in malware analysis with machine learning, which is crucial for real-world applications, especially where explainability is key.

6. CONCLUSION

Our research has demonstrated the importance of integrating SHAP-based feature selection into ensemble classifiers to augment classification accuracy. Through rigorous experimental validation, we have achieved a significant improvement in accuracy with our ensemble model, surpassing both the baseline ensemble's accuracy and the average accuracy of individual classifiers. This finding reaffirms the efficacy of leveraging SHAP explanations to identify and prioritise influential features in malware classification, as evidenced in other studies.

However, this research work has also shed light on issues like limited availability of data in this domain resulting into imbalanced datasets and its associated challenges. This realisation opens up several directions for future work, necessitating innovative approaches to address these constraints. For example, we have plans to experiment this approach with the combined custom dataset features from both static and dynamic malware analysis. We also intend to analyse the computational resource efficiency of the dynamic feature selection approach as it scales/adapts to large datasets in malware classification research.

ACKNOWLEDGMENTS

This work was supported by National Research Foundation (NRF) grant (No. NRF-2022R1A4A1032361) funded by the Korea government (MSIT), and Institute of Information Communications Technology Planning Evaluation (IITP) grant funded by the Korea government (MSIT) (No.2021-0-00590).

REFERENCES

- [1] 2024. Welcome to the SHAP documentation. <https://shap.readthedocs.io/en/latest/index.html>
- [2] Mustafa Al-Fayoumi and Qasem Abu Al-Haija. 2024. XAI-PDF: A Robust Framework for Malicious PDF Detection Leveraging SHAP-Based Feature Engineering. *IAJIT* 21, 1 (2024), 128. <https://doi.org/doi.org/10.34028/iajit/21/1/12>
- [3] Ömer Aslan Aslan and Refik Samet. 2020. A Comprehensive Review on Malware Detection Approaches. *IEEE Access* 8 (2020), 6249–6271. <https://doi.org/10.1109/ACCESS.2019.2963724>
- [4] Nureni Ayofe Azeez, Oluwanifise Ebuloluwa Odufuwa, Sanjay Misra, Jonathan Oluranti, and Robertas Damaševičius. 2021. Windows PE Malware Detection Using Ensemble Learning. *Informatics* 8, 1 (2021), 10. <https://doi.org/10.3390/informatics8010010>
- [5] Zhenshuo Chen, Eoin Brophy, and Tomas Ward. 2021. Malware Classification Using Static Disassembly and Machine Learning. *arXiv preprint arXiv:2201.07649* (2021).
- [6] Adam Duby, Teryl Taylor, Gedare Bloom, and Yanyan Zhuang. 2022. Evaluating Feature Robustness for Windows Malware Family Classification. In *2022 International Conference on Computer Communications and Networks (ICCCN)*. 1–10. <https://doi.org/10.1109/ICCCN54977.2022.9868914>
- [7] Zhiyang Fang, Junfeng Wang, Jiaxuan Geng, and Xuan Kan. 2019. Feature Selection for Malware Detection Based on Reinforcement Learning. *IEEE Access* 7 (2019), 176177–176187. <https://doi.org/10.1109/ACCESS.2019.2957429>
- [8] Weijie Han, Jingfeng Xue, YongWang, Lu Huang, Zixiao Kong, and Limin Mao. 2019. MalDAE: Detecting and explaining malware based on correlation and fusion of static and dynamic characteristics. *Computers Security* 83 (2019), 208–233. <https://doi.org/10.1016/j.cose.2019.02.007>
- [9] Jaehyeong Lee, Hyuk Jang, Sungmin Ha, and Yourim Yoon. 2021. Android Malware Detection Using Machine Learning with Feature Selection Based on the Genetic Algorithm. *Mathematics* 9 (2021), 2813. Issue 21. <https://doi.org/10.3390/math9212813>
- [10] Kewei Li and Fudong Liu. 2024. ResNet and Transformer Hybrid Malware Classification Model Based on Ensemble Learning. In *Proceedings of the 2023 7th International Conference on Electronic Information Technology and Computer Engineering (Xiamen, China) (EITCE '23)*. Association for Computing Machinery, New York, NY, USA, 1269–1275. <https://doi.org/10.1145/3650400.3650614>
- [11] Shudong Li, Laiyuan Jiang, Qianqing Zhang, Zhen Wang, Zhihong Tian, and Mohsen Guizani. 2023. A Malicious Mining Code Detection Method Based on Multi-Features Fusion. *IEEE Transactions on Network Science and Engineering* 10, 5 (2023), 2731–2739. <https://doi.org/10.1109/TNSE.2022.3155187>
- [12] Scott Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. *arXiv preprint arXiv:1705.07874* (2017).
- [13] Scott Lundberg Gabriel G. Erion, and Su-In Lee. 2018. Consistent Individualized Feature Attribution for Tree Ensembles. *CoRR abs/1802.03888* (2018). *arXiv:1802.03888* <http://arxiv.org/abs/1802.03888>
- [14] Packt Publishing. 2018. Mastering Machine Learning for Penetration Testing. <https://github.com/PacktPublishing/Mastering-Machine-Learning-for-Penetration-Testing>.
- [15] Kshitiz Aryal Maanank Gupta Quincy Card, Daniel Simpson and Sheikh Rabiul Islam. 2024. Explainable Deep Learning Models for Dynamic and Online Malware Classification. *arXiv preprint arXiv:2404.12473* (2024).
- [16] Charles Nicholas Ran Liu, Maksim Eren. 2023. Can Feature Engineering Help Quantum Machine Learning for Malware Detection? *arXiv* (2023). <https://doi.org/10.48550/arXiv.2305.0239>
- [17] Mohd Faizal Ab Razak, Nor Badrul Anuar, Rosli Salleh, and Ahmad Firdaus. 2016. The rise of “malware”: Bibliometric analysis of malware study. *Journal of Network and Computer Applications* 75 (2016), 58–76. <https://doi.org/10.1016/j.jnca.2016.08.022>
- [18] Cho San and Mie Su Thwin. 2019. Proposed Effective Feature Extraction and Selection for Malicious Software Classification. Springer International Publishing, Cham, 51–71. https://doi.org/10.1007/978-3-030-30436-2_3
- [19] Ji Shouling, Li Jinfeng, Du Tianyu, and Li Bo. 2019. Survey on Techniques, Applications and Security of Machine Learning Interpretability. *Journal of Computer Research and Development* 56, 10 (2019), 2071–2096. <https://doi.org/10.7544/issn1000-239.2019.20190540>
- [20] Jarne Verhaeghe, Jeroen Van Der Donckt, Femke Ongenaes, and Sofie Van Hoecke. 2023. Powershap: A Power-Full Shapley Feature Selection Method. In *Machine Learning and Knowledge Discovery in Databases*, Massih-Reza Amini, Stephane Canu, Asja Fischer, Tias Guns, Petra Kralj Novak, and Grigorios Tsoumakas (Eds.). Springer International Publishing, Cham, 71–87. https://doi.org/10.1007/978-3-031-26387-3_5
- [21] Lu Xiaofeng. 2018. ASSCA: API based Sequence and Statistics features Combined malware detection Architecture. *Procedia Computer Science* 129 (2018), 248–256. <https://doi.org/10.1016/j.procs.2018.03.072>
- [22] Yuntao Zhao, Bo, Yongxin Feng, ChunYu Xu, and Bo Yu. 2019. A feature extraction method of hybrid gram for malicious behavior based on machine learning. *Secur. Commun. Netw.* (2019). <https://doi.org/10.1155/2019/2674684>
- [23] KONG Zixiao, XUE Jingfeng, WANG Yong, ZHANG Qian, HAN Weijie, and ZHU Yufen. 2023. MalFSM: Feature Subset Selection Method for Malware Family Classification. *Chinese Journal of Electronics* 32, 1 (2023), 26–38. <https://doi.org/10.23919/cje.2022.00.038>
- [24] Erik Štrumbelj and Igor Kononenko. 2014. Explaining prediction models and individual predictions with feature contributions. *Knowledge and Information Systems* 41, 3 (2014), 647–665. <https://doi.org/10.1007/s10115-013-0679-x>